

■ CRA TEAM 3 · CTRL-C

CodeFab

새로운 확장자 `.ctrlc`로 작성한 코드를
분석하고 실행하는 Interpreter

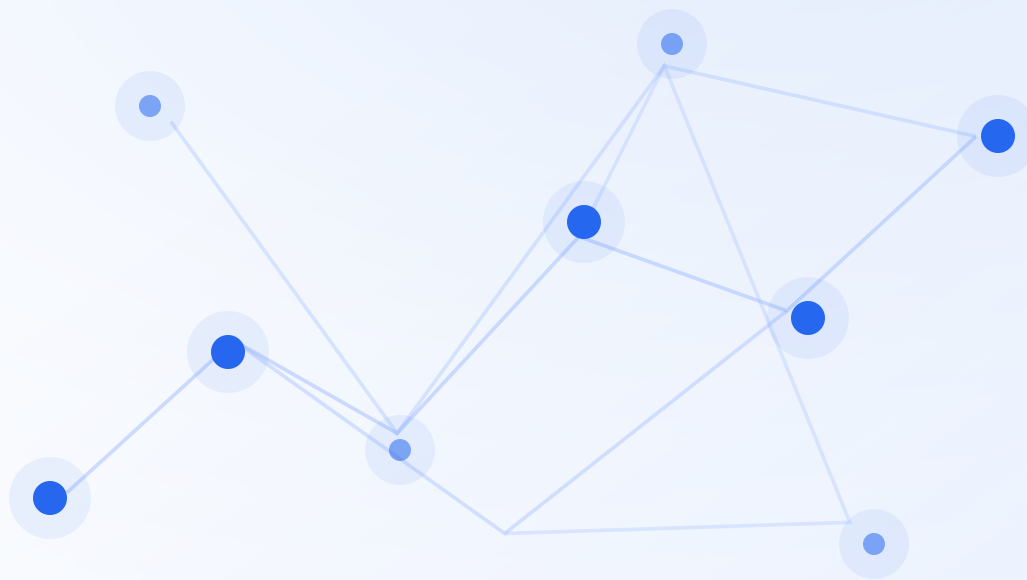
Language

Pipeline

Shell

Design Patterns

Collaboration



목차

언어 구현과 협업 과정을 함께 봅니다

기본 요구사항을 어떤 구조로 실행했고, 기능 확장과 설계 패턴을 어디에 적용했는지 순서대로 설명합니다.

01

팀 소개

역할과 구현 분담

02

프로젝트 목표

새로운 확장자 `.ctrlc`

03

전체 구조

Shell과 interpreter
pipeline

04

주요 기능

문법, 함수, class, import,
checker

05

디자인 패턴

AST, Callable,
Command 구조

06

협업과 품질

Review, TDD, Actions,
protection

■ TEAM

역할을 나누고, PR 단위로 하나의 Interpreter에 합쳤습니다

김명준

팀장

architecture, infra,
CI, coverage,
document

권소영

assembler,
executor, function,
class, import

김민준

expression,
statement, shell,
file/debug mode

박진우

tokenizer, array,
custom language

이예진

checker,
optimizer, custom
function

■ PROJECT GOAL

새로운 확장자 `.ctrlc`를 실행 가능한 언어로 만듭니다

입력

- `.ctrlc` source file
- prompt에서 입력한 한 줄 코드
- debug 대상으로 실행할 프로그램



결과

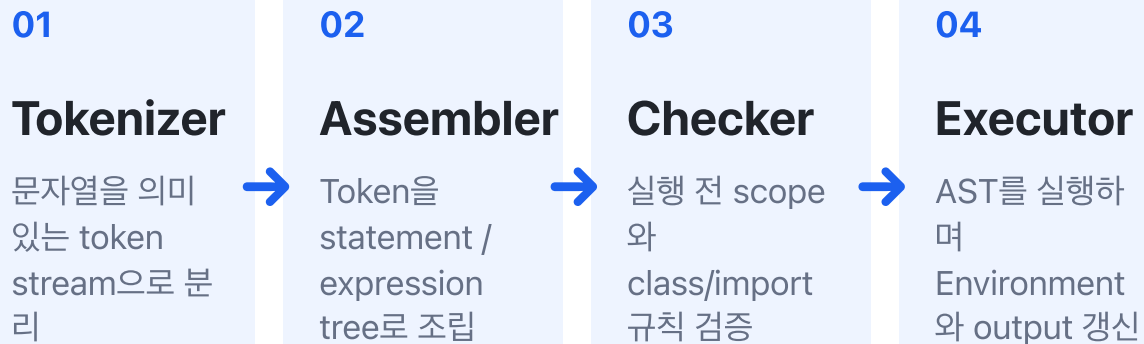
- source를 Token과 Expr/Stmt AST로 구조화
- Checker로 scope, class, import 규칙을 실행 전에 확인
- Executor가 Environment를 갱신하며 출력과 오류를 생성

Shell은 실행 방식을 나누고, Pipeline은 같은 순서로 해석합니다

Shell Modes

- 01 **Prompt Mode**
한 줄 입력을 즉시 실행
- 02 **File Mode**
`.ctrlc` 파일 전체 실행
- 03 **Debug Mode**
Stmt 단위로 실행 상태 확인

Common Interpreter Pipeline



01

■ FEATURE IMPLEMENTATION

주요 기능은 **실행 흐름** 기준으로 설명합니다

각 기능은 무엇을 입력받고, 내부에서 어떤 구조로 바뀌며, 실행 결과에 어떤 영향을 주는지 같은 방식으로 봅니다.

Basic

기본 문법

Stmt와 Expr로
프로그램 흐름
을 표현

Core

함수와 Class

Callable,
This, Super로
실행 단위 확장

Module

Array와 Import

데이터와 파일
단위 재사용 지
원

Safety

Checker

실행 전 scope
와 문맥 오류 검
증

Shell

실행 모드

prompt, file,
debug를 같은
pipeline에 연
결

기본 문법은 **Stmt**와 **Expr**로 나뉘어 실행됩니다

구현 구조

실행 순서를 만드는 문장은 Stmt, 값을 계산하는 식은 Expr로 분리해 같은 pipeline에서 다룹니다.

01 Tokenizer

예약어, 식별자, 연산자를 token으로 분리

02 Assembler

조건문과 반복문을 Stmt tree로 조립

03 Executor

Stmt는 순서대로 실행하고 Expr은 값으로 평가

[app/assembler/statement.py](#) · [app/assembler/expr.py](#)

```
var total = 0;

for (var i = 0; i < 3; i = i + 1) {
    total = total + i;
}

if (total > 2) {
    print total;
}
```

함수는 선언 시 저장되고, 호출 시 새 **scope**에서 실행됩니다

```
Func add(a, b) {  
    return a + b;  
}  
  
print add(2, 3);
```

호출 흐름

함수 본문은 선언 시 실행되지 않고, 호출되는 순간 closure를 부모로 둔 Environment에서 실행됩니다.

01 FunctionStmt

함수 이름, parameter, body를 AST에 저장

02 UserFunction

선언과 closure를 함께 보관

03 call()

argument를 parameter에 묶고 return 값을 호출 지점으로 전달

[app/assembler/runtime.py](#)

객체 실행 흐름

Class를 Callable로 맞춰 함수 호출과 같은 경로에서 instance 생성, method binding, 상속 탐색을 처리합니다.

01 UserClass

class name, method table, superclass 저장

02 UserInstance

field를 보관하고 method lookup을 class에 위임

03 This binding

method 호출 전 현재 instance를 `This`에 연결

04 Super lookup

상위 class에서 method를 찾아 현재 instance로 식해

```
Class Robot {  
    move() { print "move"; }  
}  
  
Class SpeedRobot : Robot {  
    move() {  
        Super.move();  
        print "speed";  
    }  
}
```

Array와 Import는 데이터를 담고, 파일 단위로 기능을 재사용합니다

Static Array

`Array(size)` 로 고정 크기 list를 만들고, index 접근은 Executor가 타입과 범위를 확인합니다.

```
Array(3)
```

```
arr[0] = value
```

```
IndexSetExpr / IndexGetExpr
```

Library Import

다른 `.ctrlc` 파일을 같은 Tokenizer와 Assembler로 읽고, alias namespace를 통해 선언에 접근합니다.

```
import "math.ctrlc" alias math
```

Tokenizer → Assembler 재사용

```
math.add() 형태로 접근
```

Checker는 실행 전에 **줄일 것과 막을 것**을 나눕니다

ConstantFolder

리터럴끼리 계산 가능한 expression은 미리 LiteralExpr로 치환

ScopeStack

선언, 초기화 여부, 정적 scope distance를 추적

Resolver

Stmt와 Expr를 나눠 This, Super, return, import 규칙을 확인

Scope

중복 선언, 미초기화 변수, alias 충돌

Class

This/Super 위치, 자기 상속, init return

Import

중복 import, 순환 import, 허용 선언 제한

Report

실행 전 오류를 source line 기준으로 전달

정적 바인딩은 변수 위치를 실행 전에 계산합니다

왜 필요한가

중첩 scope가 깊어질수록 실행 중 이름을 계속 탐색하면 비용과 동작 예측이 어려워집니다.

Checker가
선언 위치를
먼저 찾음

Expr node
에 distance
저장

Executor는
저장된 거리로
바로 접근

[app/checker/scope_stack.py](#) ·
[app/executor/environment.py](#)

VariableExpr(name)

ScopeStack.distance_to(name)

expr.distance = depth

Environment.get_at(distance, name)

Shell은 세 가지 모드를 같은 실행기로 연결합니다

실행 모드

01 Prompt Mode
한 줄 입력을 즉시 실행

02 File Mode
`.ctrlc` 파일 전체 실행

03 Debug Mode
breakpoint, step, watch로 상태 확인

Prompt 명령 처리

source line

ShellCommandFactory.create(source)

Command.execute(shell)

CodeFabRunner → common pipeline

[app/shell/commands.py](#) · [app/shell/shell.py](#)

Custom Language Features

확장 방향

교안 기반 interpreter 위에 팀 전용 문법과 발표용 함수를 엮어, 같은 pipeline 안에서 처리되도록 만들었습니다.

한글 alias

영어 keyword와 같은 token type으로 정규화해 실행 흐름을 공유합니다.

ctrl_c()

팀 소개와 역할을 출력하는 custom native function입니다.

♡ 연산

문자열 피연산자를 받아 이름 궁합 점수를 계산합니다.

.ctrlc

파일 모드와 import 기준이 되는 팀 전용 확장자입니다.

02

■ DESIGN PATTERNS

구현에 적용한 디자인 패턴

반복되는 구현 문제를 어떤 구조로 정리했는지 패턴별로 연결해서 설명합니다.

디자인 패턴은 복잡도가 커지는 지점에 적용했습니다

AST 구조

Composite-style AST

Stmt와 Expr node가 서로를 포함하는 tree가 되어 Checker와 Executor가 같은 구조를 따라갑니다.

expr.py · statement.py

호출 규칙

Callable Adapter

native 함수, 사용자 함수, class 생성자를 모두 `arity()`와 `call()` 규칙으로 맞춥니다.

runtime.py · executor.py

생성 책임

Factory-like Creation

source 입력은 Tokenizer와 Assembler를 지나 token과 AST node로 만들어집니다.

tokenizer.py · assembler.py

Shell 명령

Command Pattern

prompt 입력을 command 객체로 바꾸고, Shell은 `execute(shell)` 만 호출합니다.

commands.py · shell.py

Composite-style AST: 코드를 트리 구조로 다룹니다

적용 지점

조건문, 반복문, 함수 본문처럼 내부에 다른 문장과 식이 들어가는 구조를 Stmt/Expr node tree로 표현했습니다.

문법 추가 위치가 명확함

Checker와 Executor가 같은 AST를 사용함

중첩 구조를 자연스럽게 표현함

[app/assembler/expr.py](#) · [app/assembler/statement.py](#)

ForStmt

initializer: VarStmt

condition: BinaryExpr

increment: AssignExpr

body: BlockStmt([...])

Callable Adapter: 서로 다른 호출 대상을 **같은 규칙**으로 실행합니다

`Executor.evaluate_call()`

`callee.arity()`

`callee.call(executor, args)`

`NativeFunction · UserFunction · UserClass`

적용 지점

내장 함수, 사용자 함수, class 생성은 내부 동작이 다르지만 Executor는 Callable 규칙만 알면 됩니다.

호출 검증 로직을 한 곳에 둬

class 생성도 함수 호출처럼 처리

native function 확장이 쉬움

[app/assembler/runtime.py](#) · [app/executor/executor.py](#)

Factory-like Creation: source를 실행 가능한 객체 구조로 만듭니다

적용 지점

엄격한 Factory Method라기보다는, token과 AST node 생성 책임을 Tokenizer와 Assembler에 모은 생성 구조입니다.

문법 추가 경
로가 일정함

import도 같
은 parser 호
를 재사용

source와 실행
구조를 분리

[app/assembler/tokenizer.py](#) ·
[app/assembler/assembler.py](#)

source text

Tokenizer.tokenize()

Token[]

Assembler.parse()

Stmt[] / Expr tree

Command Pattern: Shell 명령을 독립 실행 단위로 분리했습니다

PromptShell.run_line(source)

ShellCommandFactory.create(source)

**Exit · Recommend · Rerun · Explain ·
RunCode**

command.execute(shell)

적용 지점

PromptShell이 모든 명령 분기를 직접 처리하지 않고, 입력을 Command 객체로 바꾼 뒤 실행만 위임합니다.

명령별 실행
책임 분리

history 저장
여부를
command
가 결정

새 shell 명령
추가 범위 축
소

[app/shell/commands.py](#) · [app/shell/shell.py](#)

03

■ COLLABORATION

협업 방식과 품질 기준

코드 리뷰, 테스트 자동화, 보호 규칙을 통해 main에 들어가는 변경의 기준을 맞췄습니다.

Code Review 사례

의도 공유

PR template로 변경 이유와 검증 범위를 먼저 맞췄습니다.

관점 확보

최소 2명 approve 기준으로 merge 전 확인 관점을 늘렸습니다.

수정 근거

debug 출력, import 경로, checker 누락을 리뷰로 보강했습니다.

TDD와 회귀 테스트 사례

실패 고정

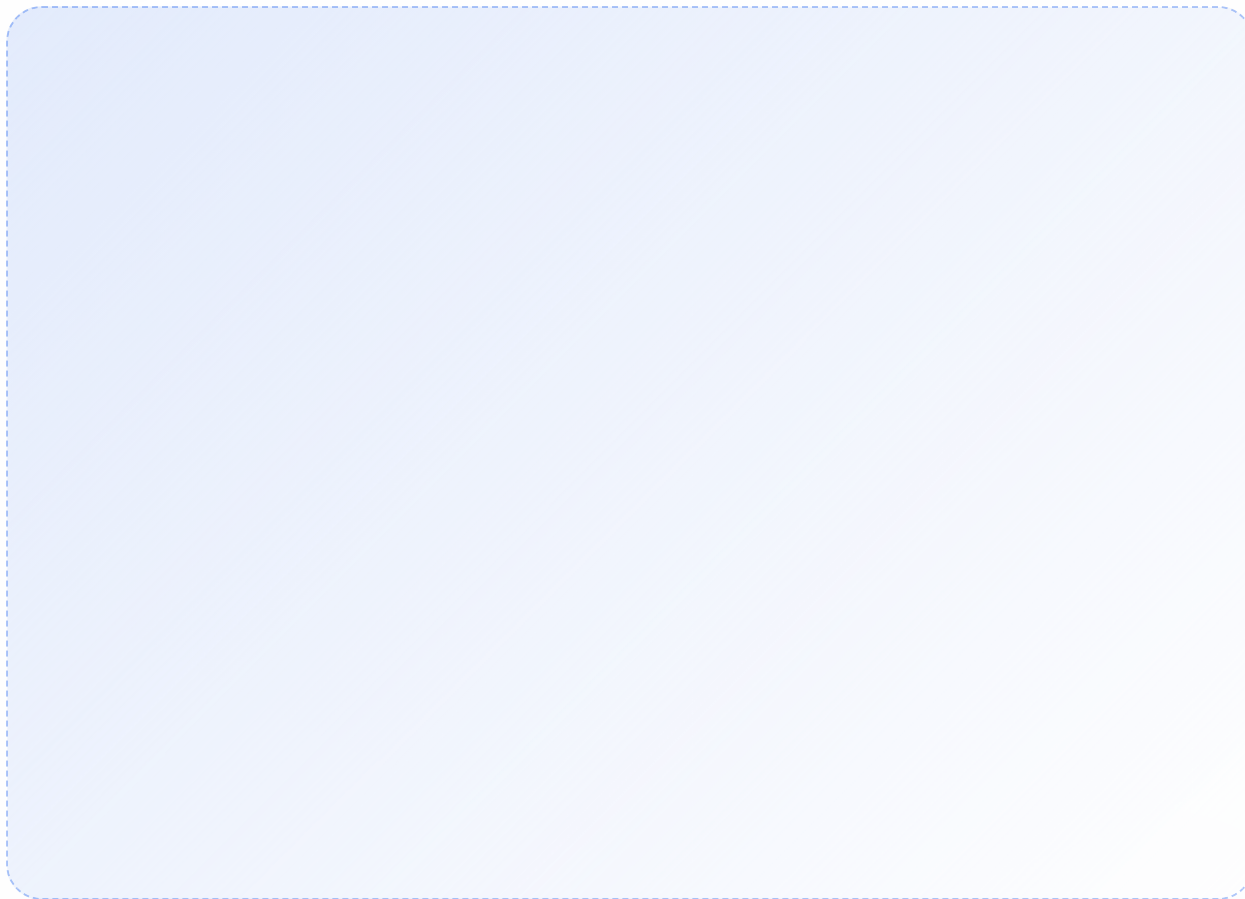
기능 구현 전에 깨져야 하는 케이스를 테스트로 먼저 남겼습니다.

회귀 방지

import, debug, checker 오류를 반복 가능한 테스트로 관리했습니다.

자동 검증

GitHub Actions가 PR마다 같은 기준으로 테스트를 실행했습니다.



Shell Command Refactoring

Before

- PromptShell 안에서 명령 분기가 커짐
- 기능 추가 시 기존 흐름을 계속 건드림
- 명령별 테스트 범위가 넓어짐

After

- 명령을 Command 객체로 분리
- PromptShell은 command 선택과 실행만 담당
- 새 shell 기능을 독립 단위로 추가

PR은 같은 흐름으로 작성하고 검토했습니다

01

Branch

작은 기능 단위로 작업 범위를 나눕니다.



02

PR Template

변경 요약, 검증 결과, self-review를 기록합니다.



03

Review

재현 가능한 예시와 테스트 추가 여부를 확인합니다.



04

Merge

approve와 required check 이후 main에 반영합니다.

Actions와 Protection Rule

GitHub Actions

- black, isort, ruff로 코드 스타일 검증
- pytest와 coverage report 실행
- gist acceptance로 대표 언어 흐름 확인

Protection Rule

- main 직접 반영 방지
- approve 후 merge
- required check 통과 후 반영